Universidad Autónoma de Yucatán

Facultad de Matemáticas

Ejercicio 1: Q-learning en Cliff Walking

Inteligencia Artificial

Fernando Alberto Loza Sánchez

6 de noviembre de 2026

Índice

1. El problema en pocas palabras **2**

2. El código **2**
2.1. El mundo 2
2.2. La tabla Q y cómo elijo acciones 3
2.3. El entrenamiento (aquí está Q-learning) 4
2.4. Evaluar la política greedy 5
2.5. Cómo se corre 5

3. La política aprendida **6**

4. La recompensa por episodio **6**

5. El camino de los 13 pasos es óptimo **7**

6. La curva de entrenamiento queda por debajo de -13 aunque la política greedy ya sea óptima **8**

7. s' en el backup de Q-learning, y por qué ese backup no trata el acantilado como estado terminal **8**

1. El problema en pocas palabras

Es el *Cliff Walking* del libro de Sutton y Barto [1], tal como lo pide el ejercicio 1 de Q-learning del curso [2]. Una cuadrícula de 4×12; la fila 0 es la de arriba:

fila\col	0	1	2	3	4	5	6	7	8	9	10	11
0	.	.	.	.	.	.	.	.	.	.	.	.
1	.	.	.	.	.	.	.	.	.	.	.	.
2	.	.	.	.	.	.	.	.	.	.	.	.
3	S	C	C	C	C	C	C	C	C	C	C	G

Lo que hay que tener en la cabeza:

- Empiezo en S $= (3, 0)$ y quiero llegar a G $= (3, 11)$.
- Puedo moverme N, E, S, W. Si me saldría del mapa, me quedo donde estoy.
- **Cada paso cuesta** -1 (también el que entra a G y el que choca con el borde).
- Las casillas C son el acantilado. **Si piso una, me cobran** -100 **y me regresan a S**, pero el episodio **sigue**.
- El episodio solo se acaba al entrar a G (o si llego al tope de 200 pasos).
- Sin descuento: $\gamma = 1$. O sea, el retorno es simplemente la suma de todo lo que me cobraron.

Parámetros de la corrida: $\alpha = 0{,}5$, $\gamma = 1$, $\varepsilon = 0{,}1$, 500 episodios, Q inicial en 0 y **semilla = 0**. El desempate es por el orden fijo N, E, S, W (gana la primera de la lista).

2. El código

Todo está en un solo archivo, `cliff_qlearning.py`, en Python puro (sin NumPy ni Gym). Lo pongo por pedazos y explico qué hace cada uno.

2.1. El mundo

Un estado es una pareja `(fila, columna)`. La función `dar_un_paso` es *todo* el entorno: le doy dónde estoy y qué acción hago, y me dice a dónde llego, cuánto me cobran y si se acabó el episodio.

```
FILAS = 4
COLUMNAS = 12

INICIO = (3, 0) # S
META = (3, 11) # G

# El acantilado: la fila de abajo, de la columna 1 a la 10
ACANTILADO = []
for col in range(1, 11):
    ACANTILADO.append((3, col))

# Cada accion dice cuanto se mueve (fila, columna).
# OJO: la fila 0 es la de ARRIBA, por eso "N" resta 1 a la fila.
ACCIONES = ["N", "E", "S", "W"] # este orden tambien desempata
MOVIMIENTO = {
    "N": (-1, 0),
    "E": (0, +1),
    "S": (+1, 0),
    "W": (0, -1),
}

def dar_un_paso(estado, accion):
```

```
    """Regresa (siguiente_estado, recompensa, termino_el_episodio)."""
    fila, col = estado
    df, dc = MOVIMIENTO[accion]
    nueva_fila = fila + df
    nueva_col = col + dc

    # Si me saldria del mapa, me quedo donde estoy
    if nueva_fila < 0 or nueva_fila >= FILAS:
        nueva_fila = fila
    if nueva_col < 0 or nueva_col >= COLUMNAS:
        nueva_col = col

    nuevo = (nueva_fila, nueva_col)

    if nuevo in ACANTILADO:
        # Me cai: -100 y regreso a S. El episodio NO termina.
        return INICIO, -100, False
    if nuevo == META:
        # Llegue: -1 y se acaba el episodio.
        return META, -1, True
    # Paso normal (incluye chocar con el borde)
    return nuevo, -1, False
```

Lo importante está en los dos `if` del final:

- Caer al acantilado regresa `(INICIO, -100, False)`: el siguiente estado es S y el `False` dice "esto **no** se acabó".
- Entrar a G regresa `(META, -1, True)`: aquí sí se acaba.

Antes de entrenar compruebo a mano los tres casos que pide el enunciado. Si alguno falla, el programa truena ahí mismo:

```
assert dar_un_paso(INICIO, "N") == ((2, 0), -1, False) # subo, -1
assert dar_un_paso(INICIO, "E") == (INICIO, -100, False) # me caigo, vuelvo a S, sigue
assert dar_un_paso((2, 11), "S") == (META, -1, True) # entro a G, se acaba
```

2.2. La tabla Q y cómo elijo acciones

La tabla Q es un diccionario de diccionarios: `Q[estado][accion]` es un número. Empieza todo en 0. La meta *no* lleva fila porque desde ahí ya no se hace nada. Las casillas del acantilado sí tienen fila, pero nunca se usan: el agente jamás se queda parado en una C, porque en cuanto la pisa lo mandan a S.

```
def tabla_q_vacia():
    """Q[estado][accion] = 0 para todo estado que no sea la meta."""
    Q = {}
    for fila in range(FILAS):
        for col in range(COLUMNAS):
            estado = (fila, col)
            if estado == META:
                continue # la meta es terminal: no lleva fila en Q
            Q[estado] = {}
            for a in ACCIONES:
                Q[estado][a] = 0.0
    return Q
```

Para elegir acción hay dos funciones:

- `mejor_accion`: la acción con el Q más grande. Recorro N, E, S, W y solo cambio de opinión si encuentro una *estrictamente* mayor (`>`); por eso en un empate gana la que aparece primero.
- `elegir_accion`: la ε-greedy. Tiro un número al azar en $[0, 1)$; si es menor que ε, exploro (acción al azar entre las 4); si no, uso la mejor.

```python
def mejor_accion(Q, estado):
    """La accion con mayor Q. En empate gana la primera de N, E, S, W."""
    mejor = ACCIONES[0]
    for a in ACCIONES[1:]:
        if Q[estado][a] > Q[estado][mejor]: # '>' estricto: no cambia en empate
            mejor = a
    return mejor

def elegir_accion(Q, estado, epsilon, rng):
    """epsilon-greedy: con prob. epsilon al azar; si no, la mejor."""
    if rng.random() < epsilon:
        return rng.choice(ACCIONES)
    return mejor_accion(Q, estado)
```

Ojo con un detalle: cuando exploro, la acción al azar puede salir *igual* a la mejor. Así que la probabilidad de hacer una acción *específica* no greedy (por ejemplo "S" encima del acantilado) es $\varepsilon/4 = 0{,}025$ por paso.

2.3. El entrenamiento (aquí está Q-learning)

```python
def entrenar(episodios, alpha, gamma, epsilon, semilla, tope_pasos=200):
    rng = random.Random(semilla)
    Q = tabla_q_vacia()
    recompensa_por_episodio = []

    for _ in range(episodios):
        s = INICIO
        total = 0 # lo que junto en este episodio

        for _paso in range(tope_pasos):
            a = elegir_accion(Q, s, epsilon, rng) # lo que HAGO
            s_sig, r, termino = dar_un_paso(s, a)
            total += r

            # Lo que "espero" del futuro: el maximo de Q en s_sig.
            # Si llegue a la meta, ya no hay futuro -> 0.
            # Si me cai, s_sig es S, que es un estado normal -> max Q(S, .)
            if termino:
                futuro = 0.0
            else:
                futuro = max(Q[s_sig].values())

            objetivo = r + gamma * futuro
            error = objetivo - Q[s][a]
            Q[s][a] = Q[s][a] + alpha * error # el backup de Q-learning

            s = s_sig
            if termino:
                break

        recompensa_por_episodio.append(total)

    return Q, recompensa_por_episodio
```

Leído en español, cada paso hace esto:

1. Estoy en `s`. Elijo `a` con ε-greedy (a veces exploro).
2. Le pido al mundo el paso: me da `s_sig`, `r` y si `termino`.
3. Calculo el **objetivo**: lo que me acaban de cobrar más lo mejor que *creo* que puedo sacar desde `s_sig`:
$$\text{objetivo} = r + \gamma \max_{a'} Q(s', a').$$
Si `s_sig` es la meta, el futuro vale 0.

4. Muevo $Q(s,a)$ un poquito (la mitad, porque $\alpha = 0{,}5$) hacia ese objetivo:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[\text{objetivo} - Q(s,a)\right].$$

5. Me paso a `s_sig` y repito.

Lo que hace que esto sea Q-learning (y no Sarsa) es el `max` del paso 3: el objetivo usa la *mejor* acción en s', aunque después el agente, por explorar, no vaya a hacer esa acción. La acción que *hago* sale de la ε-greedy; la que *uso para aprender* es la greedy. A eso le dicen *off-policy*.

La variable `total` va sumando las recompensas del episodio *con la exploración prendida*; esa es la serie que grafico en la sección 4.

2.4. Evaluar la política greedy

Ya entrenada la tabla, camino desde S siempre con `mejor_accion` (o sea $\varepsilon = 0$) y anoto el camino, cuántos pasos di y cuánto me cobraron:

```
def recorrer_greedy(Q, tope_pasos=200):
    s = INICIO
    camino = [s]
    acciones = []
    total = 0
    for _ in range(tope_pasos):
        a = mejor_accion(Q, s)
        s, r, termino = dar_un_paso(s, a)
        acciones.append(a)
        camino.append(s)
        total += r
        if termino:
            break
    return camino, acciones, total
```

2.5. Cómo se corre

```
SEMILLA = 0
ALPHA, GAMMA, EPSILON, EPISODIOS = 0.5, 1.0, 0.1, 500

Q, recompensas = entrenar(EPISODIOS, ALPHA, GAMMA, EPSILON, SEMILLA)
camino, acciones, retorno = recorrer_greedy(Q)
```

En la terminal: `python3 cliff_qlearning.py`. Imprime la política, el camino y el retorno, y guarda la recompensa de cada episodio en `recompensas.csv`. La salida que obtuve fue:

```
Semilla: 0
Politica greedy:
> > > > > > > v > > v v
v v > > > > v > > v > v
> > > > > > > > > > > v
S C C C C C C C C C G
Acciones desde S: N E E E E E E E E E S
Pasos: 13 Retorno greedy: -13
```

3. La política aprendida

La política greedy es: en cada casilla, haz la acción con mayor Q. Aquí está dibujada con flechas; en verde el camino que de verdad sigue el agente desde S.

	0	1	2	3	4	5	6	7	8	9	10	11
0	→	→	→	→	→	→	→	↓	→	→	↓	↓
1	↓	↓	→	→	→	→	↓	→	→	↓	→	↓
2	→	→	→	→	→	→	→	→	→	→	→	↓
3	S↑	C	C	C	C	C	C	C	C	C	C	G

Cómo leerla:

- **En S la flecha es** ↑: lo primero es subir a $(2, 0)$.
- **Toda la fila 2 apunta a la derecha** hasta la columna 11, que apunta hacia abajo, a G.
- El camino completo es

$$S \xrightarrow{N} (2,0) \xrightarrow{E} (2,1) \xrightarrow{E} \cdots \xrightarrow{E} (2,11) \xrightarrow{S} G,$$

  es decir `N`, once veces `E`, y `S`: **13 pasos**, cada uno de -1, así que el **retorno greedy es** -13. Va pegado al acantilado, como dice el libro.
- Las filas 0 y 1 casi no se visitan, pero aun así todas sus flechas son → o ↓: desde cualquiera de esas casillas la política también llega a G sin dar pasos de más.

Para ver por qué gana "derecha" sobre "abajo" en la fila 2, estos son algunos valores de Q al final del entrenamiento:

estado	$Q(\cdot, \mathrm{N})$	$Q(\cdot, \mathrm{E})$	$Q(\cdot, \mathrm{S})$	$Q(\cdot, \mathrm{W})$
S $= (3, 0)$	$\mathbf{-13{,}00}$	$-113{,}00$	$-14{,}00$	$-14{,}00$
$(2, 1)$	$-12{,}98$	$\mathbf{-11{,}00}$	$-112{,}99$	$-13{,}00$
$(2, 5)$	$-9{,}00$	$\mathbf{-7{,}00}$	$-112{,}98$	$-9{,}00$
$(2, 10)$	$-4{,}00$	$\mathbf{-2{,}00}$	$-112{,}46$	$-4{,}00$

En negritas la acción greedy. Se lee directo: desde $(2, 5)$, ir a la derecha cuesta 7 pasos más (-7); bajar es caerse (-100) y volver a empezar desde S (-13), casi -113.

4. La recompensa por episodio

Aquí "recompensa del episodio" es la suma de todo lo que me cobraron en ese episodio de entrenamiento, **con** $\varepsilon = 0{,}1$ **prendido**. La curva clara es la cruda (brinca mucho); la azul es el promedio de los últimos 10 episodios; la roja punteada es el -13 de la política greedy.

Lo mismo en números, promediando por bloques de 50 episodios:

episodios	promedio	episodios	promedio
1–50	$-100{,}7$	251–300	$-47{,}7$
51–100	$-27{,}8$	301–350	$-47{,}2$
101–150	$-45{,}2$	351–400	$-54{,}5$
151–200	$-33{,}9$	401–450	$-45{,}1$
201–250	$-37{,}6$	451–500	$-52{,}3$

Qué se ve:

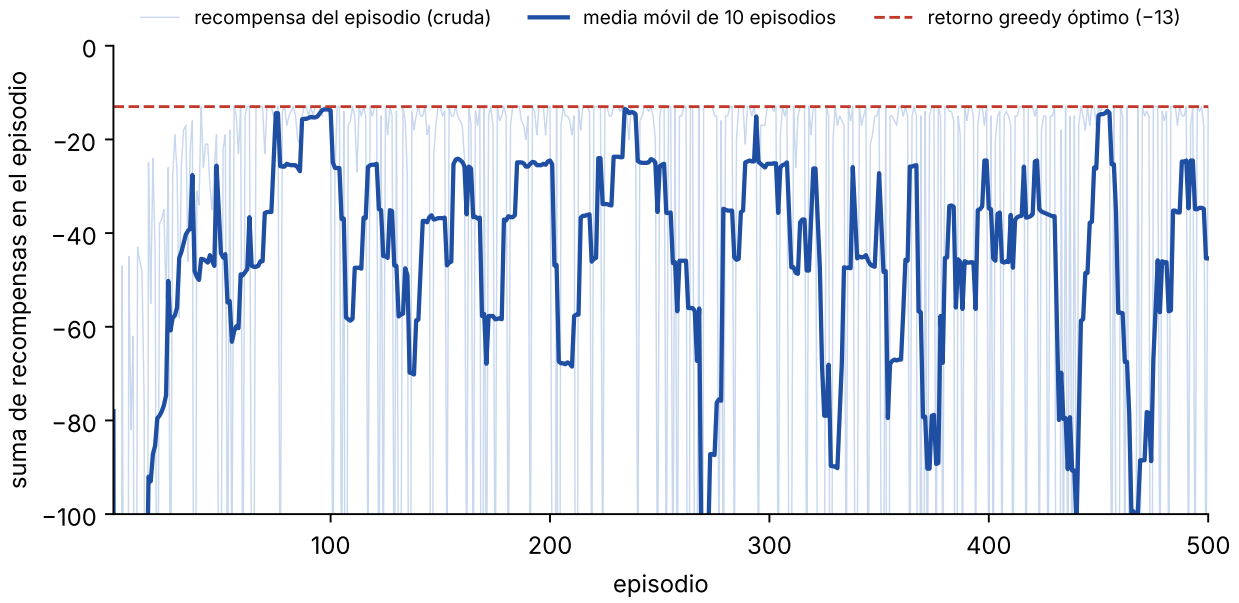


Figura 1: Recompensa por episodio en los 500 episodios (semilla 0). El eje vertical está recortado en -100; los primeros episodios llegan hasta -893.

- **Al principio es muy malo** (el episodio 2 da -893: el agente todavía no sabe nada, da vueltas y se cae varias veces antes de llegar a G).
- **Hacia el episodio 50 ya aprendió el camino.** A partir de ahí, en un episodio sin caídas junto entre -13 y unos -17. De hecho, 171 episodios dieron exactamente -13.
- **Pero la curva no se pega a** -13: se queda oscilando alrededor de -45 (el promedio de los episodios 101 a 500 es $-45{,}4$). Los "picos hacia abajo" son los episodios en los que me caí: en los episodios 101–500 hubo 91 con al menos una caída.

La serie completa (los 500 valores y su media móvil) está en `recompensas.csv`.

5. El camino de los 13 pasos es óptimo

Como $\gamma = 1$, el retorno de un episodio es la suma de lo que me cobran. Si no me caigo, cada paso vale -1, así que

$$\text{retorno} = -(\text{número de pasos}).$$

Entonces **maximizar el retorno es lo mismo que llegar a G en el menor número de pasos sin caerme**. Caerse nunca conviene: solo esa transición ya cuesta -100, y además te regresa a S, así que cualquier camino con caída da a lo más $-100 - 13 = -113$, mucho peor que -13.

¿Por qué no se puede en menos de 13?

- Para ir de la columna 0 a la 11 necesito **al menos 11 pasos E**: cada paso cambia la columna en a lo más 1.
- No puedo caminar por la fila 3 (son las C), así que tengo que **subir al menos una vez** (un N) y **bajar al menos una vez** (un S) para entrar a G.
- Total: al menos $11 + 1 + 1 = 13$ pasos.

¿Y por qué justo ese camino? Un camino de 13 pasos tiene exactamente 11 E, un N y un S, y nada más. El N tiene que ser el primero: desde S, ir a E es caer, y W o S me dejan en S (paso desperdiciado). El S tiene que ser el último: bajar desde $(2, c)$ con $1 \le c \le 10$ es caer,

y bajar en $(2, 0)$ me regresa a S. Entonces el único camino es: N, once E por la fila 2, y S. Es el que va pegado al acantilado, y es justo el que aprendió el agente.

6. La curva de entrenamiento queda por debajo de -13 aunque la política greedy ya sea óptima

Porque **la curva no mide la política greedy**: mide lo que el agente hizo mientras entrenaba, y mientras entrena usa ε-greedy con $\varepsilon = 0{,}1$. Esa política "casi greedy" camina por el mismo camino, pero de vez en cuando hace una acción al azar.

El problema es *dónde* está el camino: pegado al acantilado. En cada una de las casillas $(2, 1), \ldots, (2, 10)$, y también en S (donde ir a E es caer), una acción al azar puede ser justo la que me tira. Eso pasa con probabilidad $\varepsilon/4 = 0{,}025$ por paso. Son 11 pasos de riesgo, así que

$$P(\text{caerme al menos una vez en el episodio}) \approx 1 - (1 - 0{,}025)^{11} \approx 0{,}24.$$

Más o menos uno de cada cuatro episodios trae un -100 encima. En mi corrida, de los episodios 101–500, cayeron 91 de 400 ($\approx 0{,}23$), que cuadra muy bien.

Y además de las caídas, las acciones al azar que no tiran (subir, ir a W) también agregan pasos. Por eso el promedio queda alrededor de -45, muy por debajo de -13, aunque la tabla Q ya diga el camino correcto.

Dicho de otro modo: Q-learning aprende el valor del camino *óptimo* (el que va pegado al borde), pero lo aprende *mientras explora*, y explorar pegado al borde sale caro. Si apago la exploración ($\varepsilon = 0$), el mismo Q da exactamente -13. (Sarsa, en cambio, aprende el valor de lo que de verdad hace explorando, y por eso termina eligiendo un camino más lejos del borde, que es peor con $\varepsilon = 0$ pero mejor mientras explora.)

7. s' en el backup de Q-learning, y por qué ese backup no trata el acantilado como estado terminal

Tomemos un paso que cae, por ejemplo estar en $s = (2, 5)$ y hacer $a = \text{S}$. El mundo responde

$$s' = S = (3, 0), \qquad r = -100, \qquad \text{termino} = \text{False}.$$

O sea, s' **es la casilla de inicio S, no la casilla C**. La C nunca aparece como estado: el agente no "se queda" ahí, la pisa y en ese mismo paso lo regresan a S. Entonces el backup es

$$Q\big((2,5),\text{S}\big) \leftarrow Q\big((2,5),\text{S}\big) + \alpha\Big[-100 + \gamma \max_{a'} Q(S, a') - Q\big((2,5),\text{S}\big)\Big].$$

¿Por qué no se trata como terminal? Porque en este mundo *no lo es*: el enunciado dice que el episodio sigue. Después de caer, todavía me falta caminar de S a G, y eso también me lo van a cobrar. El término $\max_{a'} Q(S, a')$ es justamente "lo que todavía me espera desde S". Si lo pusiera en 0 (como si fuera terminal), estaría diciendo que caerse cuesta solo -100 y ahí se acaba todo, y eso es mentira: estaría aprendiendo los valores de otro problema (el del `cliff.yaml` del proyecto, donde caer sí termina el episodio).

En mi tabla se ve clarito. Al final del entrenamiento $\max_{a'} Q(S, a') = Q(S, \text{N}) = -13$, y:

$$Q(S, \text{E}) = -113 = -100 + (-13),$$

que es exactamente "me caigo (-100) y luego hago el camino óptimo desde S (-13)". Lo mismo para $Q((2, 5), \text{S}) \approx -112{,}98$ y los demás pasos que caen: todos rondan -113. Si el backup tratara la caída como terminal, esos valores habrían quedado en -100.

En el código esto es simplemente que `dar_un_paso` regresa `termino = False` al caer, así que el entrenamiento entra al `else` y usa `max(Q[s_sig].values())` con `s_sig = INICIO`. El único caso en el que el futuro vale 0 es cuando entro a G.

Referencias

[1] Richard S. Sutton y Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2.ª ed. MIT Press, 2018.

[2] Víctor Uc Cetina. *Repositorio del curso de Inteligencia Artificial*. https://github.com/victoruccetina/inteligencia-artificial. 2026.